Задача A. Ёлочка

На каждом ярусе в одном ряду нечётное количество элементов мозаики. Причем в первом ярусе в ряду — 1 элемент, во втором ярусе — 3 элемента, в третьем ярусе 5 элементов и т.д. В n-м ярусе в ряду будет n-е нечётное число элементов. Чтобы найти общее количество элементов мозаики, нужно найти сумму n нечётных чисел, удвоить её и прибавить к найденному числу 2. Если сумму нечётных чисел искать в цикле, то можно пройти первую группу тестов.

Чтобы получить полный балл за задачу, нужно найти сумму n нечётных чисел по формуле. Запишем сумму n нечётных чисел в прямом и обратном порядке:

$1 + 3 + 5 + \ldots + (2 \cdot n - 1) = (2 \cdot n - 1) + (2 \cdot n - 3) + \ldots + 3 + 1.$

Теперь сложим соответствующие слагаемые этих двух сумм:

$1 + 2 \cdot n - 1 = 2 \cdot n$

$3 + 2 \cdot n - 3 = 2 \cdot n$

$\ldots$

$2 \cdot n - 1 + 1 = 2 \cdot n$

В каждой из приведенных выше сумм (исходной и записанной в обратном порядке) n слагаемых. Значит, при сложении всех слагаемых этих двух сумм мы получим $2 \cdot n \cdot n$. Мы посчитали нужную нам сумму нечётных чисел дважды. Поскольку каждый ярус выкладывается дважды, то на ёлочку нужно $2 \cdot n \cdot n$ элементов мозаики. Нужно не забыть прибавить еще два элемента мозаики для изготовления ножки ёлочки. В результате получим формулу: $2 \cdot n \cdot n + 2$.

Задача B. Разноцветные шарики

Нужно найти минимальное количество шариков, которое гарантирует наличие M шариков одного цвета.

Рассмотрим худший случай: мы вытаскиваем шарики так, чтобы как можно дольше не получить M шариков одного цвета.

Для цвета i можно безопасно вытащить не более $M - 1$ шариков. Но если шариков этого цвета всего a_i, то больше a_i вытащить нельзя. Значит, максимальное количество шариков без гарантии равно

$$\sum_{i=1}^{K} \min(a_i, M - 1).$$

После этого достаточно вытащить ещё один шарик, и по принципу Дирихле среди выбранных обязательно будет M шариков одного цвета.

Задача C. Амулет

Очевидное решение, которое позволяет пройти первую группу тестов — это перебор всех подстрок и выбор из них максимальной, в которой символы X, Y, Z не встречаются более трех раз. Сложность данного решения $\mathcal{O}(n^3)$. Перебор всех подстрок имеет сложность $\mathcal{O}(n^2)$. Для каждой подстроки еще происходит подсчет символов X, Y, Z за $\mathcal{O}(n)$.

Вторую группу тестов пройдут решения, в которых выполнена оптимизация данного перебора. Суть оптимизации следующая: если при сдвиге правой границы подстроки выясняется, что один из символов в данной подстроке встречается более трех раз, то дальнейшее рассмотрение всех подстрок с текущим левым концом и любыми большими правыми границами бессмысленно, т.к. этот символ в любом случае будет встречаться больше трех раз.

Полный балл за задачу можно получить, применив метод «двух указателей». Суть метода такова: мы будем последовательно рассматривать подстроки, задаваемые левым и правым указателями, двигая оба указателя параллельно.

Изначально левый указатель установлен в начале строки, будем рассматривать подстроки, начинающиеся с нулевого символа. Правый указатель постепенно смещается вправо, пока не встретится подстрока, в которой один из символов X, Y, Z встречается больше трех раз. Тогда мы прекращаем сдвигать правый указатель и начинаем сдвигать левый указатель вправо, пока в текущей подстроке не останется не более трех вхождений каждого из символов Y и Z. После этого вновь возвращаемся к движению правого указателя. Сложность данного решения $\mathcal{O}(n)$.

Задача D. Кто хочет стать миллионером?

Число называется достаточно круглым, если количество нулей в конце его записи составляет хотя бы половину всех цифр.

Для каждого следующего приза нужно найти минимальное достаточно круглое число, не меньшее $2 \cdot a_{i-1}$, начинаем со 100.

Идея простая, умножим текущее число на 2 и проверим, хватает ли нулей на конце, если нет, то младший ненулевой разряд обнулим и добавим к следующему 1.

Для реализации можно использовать числа в длинной арифметике. В C++ можно использовать строки или вектора цифр и написать функцию умножения на 2, удобно хранить обратную запись числа, так как при умножении может появляться новый разряд на конце.

Второй вариант, отдельно хранить счётчик нулей на конце и ненулевую часть как число, так как n не большое, ненулевая часть всегда умещается в *longlong*.

Ещё один способ, это заранее предподсчитать первые 71 число руками (на калькуляторе) записать в вектор строк и выводить первые n из них, каждое следующее можно считать не дольше минуты, тогда за час с лишним управитесь.

Задача E. Пароль от codeforces

Операция меняет местами символы в двух строках, но только в одной и той же позиции k. Значит, разные позиции не влияют друг на друга.

Пусть мы хотим сделать строку s_i равной паролю p. Рассмотрим позицию k.

- Если $s_i[k] = p[k]$, ничего делать не нужно.
- Если $s_i[k] \neq p[k]$, нужно найти другую строку s_j, где $s_j[k] = p[k]$, и поменять символы в строках i и j на позиции k.

Если для какой-то позиции такого j нет, строку s_i сделать паролем невозможно.

Количество операций для строки s_i равно количеству позиций, в которых s_i отличается от p, если все эти позиции можно исправить.

Значит, нужно перебрать строку, которую мы будем превращать в пароль, и выбрать вариант с минимальным числом несовпадений.

Алгоритм

Для каждой позиции k заранее запомним любую строку, в которой на позиции k стоит буква $p[k]$.

Затем переберём целевую строку i:

1. идём по всем позициям k;
2. если $s_i[k] \neq p[k]$, берём строку $j \neq i$ с символом $p[k]$ на позиции k; добавляем операцию (i, j, k).
3. если такой строки нет, вариант невозможен;

Важно: если $s_i[k] \neq p[k]$, то любая строка с символом $p[k]$ на позиции k автоматически отличается от i, потому что в i на этой позиции стоит другой символ.

Задача F. Выбор команды

Идея

Нужно выбрать k разных учеников и распределить их по трём ролям:

- a человек придумывают решение;
- b человек пишут код;
- c человек тестируют.

Так как $k \leqslant 15$, можно использовать динамическое программирование по количеству уже выбранных людей на каждую роль.

Пусть $dp[i][j][t]$— максимальная эффективность, если уже выбрано i решателей, j кодеров и t тестировщиков.

Изначально $dp[0][0][0] = 0$, остальные состояния равны $-\infty$.

Обрабатываем учеников по одному. Для каждого ученика можно:

- не брать его;
- взять как решателя и добавить x;
- взять как кодера и добавить y;
- взять как тестировщика и добавить z.

Чтобы один ученик не был использован несколько раз, переходы делаем в обратном порядке по индексам DP.

Формулы перехода для ученика (x, y, z):
$dp[i+1][j][t] = max(dp[i+1][j][t], dp[i][j][t] + x)$
$dp[i][j+1][t] = max(dp[i][j+1][t], dp[i][j][t] + y)$
$dp[i][j][t+1] = max(dp[i][j][t+1], dp[i][j][t] + z)$

Сложность

Число состояний не превосходит $(a+1)(b+1)(c+1)$, а сумма $a+b+c = k \leqslant 15$, поэтому состояний мало (C^2_{k+2}).

Итоговая сложность: $O(nabc)$ по времени и $O(abc)$ по памяти.

Задача G. Не подпоследовательность

Даны две последовательности A и B. Требуется найти кратчайшую последовательность C, которая не является подпоследовательностью ни A, ни B.

Идея

Будем строить последовательность C слева направо и отслеживать, где мы можем находиться в A и B, если текущий префикс C является их подпоследовательностью.

Для этого заранее посчитаем массивы переходов:

nextA$[i][x]$ = первая позиция $> i$ в A, где стоит x, или $n+1$, если такого элемента нет.

Аналогично определим nextB.

Динамика

Будем хранить массив: $dp[posA] = posB$, где $posA$ — позиция в A, до которой можно дойти, а $posB$ — максимально возможная позиция в B.

Изначально: $dp[0] = 0$.

Переход

Пусть мы находимся в состоянии $(posA, posB)$. Добавим число x:

$newA = \text{nextA}[posA][x], \quad newB = \text{nextB}[posB][x]$.

Обновляем: $dp_{new}[newA] = \max(dp_{new}[newA], newB)$.

Критерий остановки

Ищем минимальную длину $|C|$, при которой: $posA = n+1$ и $posB = m+1$.

Это означает, что C не является подпоследовательностью ни A, ни B.

Так как длина увеличивается по слоям (BFS по длине), первый найденный ответ — оптимальный.

Восстановление ответа

Запоминаем переходы:

- какой символ добавили;
- из какого состояния пришли.

После нахождения ответа восстанавливаем последовательность обратным проходом.

Особый случай

Если $k = 1$, то все элементы равны 1.

Тогда ответ: $\max(|A|, |B|) + 1$, и сама последовательность состоит из единиц.

Сложность

Предобработка: $O((n + m) \cdot k)$

Динамика: $O(L \cdot n \cdot k)$, где L — длина ответа (на практике мала, порядка $\frac{n}{k} + \frac{m}{k}$).

Память: $O((n + m) \cdot k)$.

Задача H. Собери число

Заметим, что при построении ответа нам в любой момент важен только лишь остаток от деления текущего его префикса на k. Действительно, если текущий префикс ответа имеет остаток от деления на k, равный r, то при приписывании к префиксу числа a_i этот остаток станет равный остатку от деления на k числа $r \cdot 10^{l_i} + a_i$ (l_i — количество цифр в записи числа a_i).

Тогда, очевидно, нас интересует как получать любой остаток от деления такой операцией, используя минимальное количество цифр в записи. Конечно, остаток 0 мы можем получить сразу, используя пустой префикс, поэтому для остатка 0 нас будет интересовать второй по величине ответ.

Все описанное выше позволяет нам построить граф на k вершинах (от 0 до $k-1$, соответственно остаткам), ребра в котором определяются числами a_i: из вершины v в вершину $u = (r \cdot 10^{l_i} + a_i) mod k$ длины l_i, означающее, что с помощью дописывания l_i цифр мы можем из префикса с остатком v получить префикс с остатком u. Легко заметить, что некоторые a_i в этом графе будут соответствовать одним и тем же ребрам (сейчас их nk) — это числа с одинаковой длиной десятичной записи и остатком от деления на k, поэтому стоит оставить лишь уникальные по такому критерию числа (их будет не больше $10k$), и тогда количество ребер не будет превышать $10k^2$.

Теперь всё, что от нас требуется — найти длину кратчайшего непустого пути из вершины 0 в саму себя же в построенном графе. Чтобы избежать столь неприятной цикличности, давайте просто добавим дополнительную вершину k, имеющую те же исходящие ребра, что и вершина 0, считая, что такой остаток может иметь лишь пустой префикс. Теперь задача сводится к нахождению кратчайшего пути из вершины k в вершину 0, которую можно решить алгоритмом Дейкстры за $O(k^2)$. Однако, в силу специфичности задачи, решение алгоритмом Форда-Беллмана (с правильными отсечениями, конечно) так же проходит все тесты, хоть в теории и имеет столь большие $O(k^3)$.

Восстановление ответа в задаче довольно стандартно, за исключением хранения дополнительного числа для каждой вершины, образовавшего ребро, по которому был осуществлен нужный в кратчайшем ответе переход.